

Part 21 - Production Resilience, Delivery and JVM Operations (Q273-Q292)

Twenty additional questions on topics the guide covers only briefly or not at all: cache stampedes, distributed transactions, change data capture, clocks, backpressure, Kafka failure handling, API evolution, safe deployments, connection pooling, garbage-collector choice, leak diagnosis, native images, class loading, alerting and secrets.

Every question uses the same four-tier ladder, so you can stop at the tier that matches the interview or your current level:

Tier	Purpose	Use it to
1. Foundation	Plain-language definition, key terms and a small example	Learn the topic, or check you can explain it to a non-specialist
2. Core answer	The 30-second version	Open your spoken answer
3. Follow-ups	Three escalating probes (warm-up, applied, senior), each with expected reasoning	Practise answering before reading the reasoning
4. Deep dive	Internals, a realistic failure, the trade-off and how to verify	Reach a defensible decision with evidence

Baseline: Java 21 without preview features, Spring Boot 3.x, PostgreSQL, Kafka and Kubernetes. Q289 is explicitly a Java 21 preview comparison. The two Java fragments are compiled and exercised with supplied application types in the repository's Part 21 fixture; they are sketches, not complete applications. Framework, database, cloud and Kubernetes scenarios require testing on your stack. Defaults and flags change between versions, so confirm them against the documentation for yours. Metrics, numbers and incidents are illustrative. Where no source is linked, the basis is general engineering practice.

Reviewed 9 October 2026: provider-specific cache synchronization, transaction ordering, ETag validity, retry counts, DNS caching, diagnostic impact, cooperative cancellation and scoped-value bindings are qualified below.

Group	Questions
Distributed systems and data	Q273-Q278
APIs and delivery	Q279-Q285
JVM and Java	Q286-Q290
Operations	Q291-Q292

Q283 deliberately avoids graceful shutdown, which Q19 already covers.

Distributed systems and data

Q273 What is a cache stampede and how do you handle hot keys

Tier 1 - Foundation

In plain language: a cache keeps a ready copy of something expensive to compute. A **stampede** happens when a very popular copy disappears and many requests all rebuild it at the same moment. A **hot key** is a different problem: one entry is requested so often that a single cache node cannot serve it.

Key terms:

- **Cache hit / miss:** the value was / was not found in the cache.
- **TTL (time to live):** how long an entry may be served before it expires.
- **Eviction:** the cache removing an entry to make room.
- **Request coalescing (single flight):** letting one caller rebuild a value while the others wait for its result.
- **Near-cache:** a small in-process copy in front of a shared cache.

Small example: a product page is cached for 60 seconds and receives 5,000 requests per second. When the entry expires, rebuilding takes 200 ms, so roughly 1,000 requests miss during that gap and all query the database for the same row.

Tier 2 - Core answer (30-second version)

A stampede is many simultaneous misses on one expired key, which hit the database exactly when the cache was meant to protect it. Reduce it with request coalescing, TTL jitter, serving stale data while one caller refreshes, and pre-warming. A hot key is a separate issue, where one key exceeds one node's capacity, and it needs a near-cache, key replication or splitting the value. I would verify by expiring a hot key under load and counting database queries.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does adding random jitter to TTLs help?

Expected reasoning: keys written together, for example after a deploy or a bulk warm-up, expire together, which produces a synchronized wave of misses. Jitter spreads expiry over time. It does not help a single popular key expiring on its own, which needs coalescing or early refresh.

Probe 2 (applied): You use `@Cacheable(sync = true)` on 50 instances. Is the stampede solved?

Expected reasoning: Spring delegates synchronized loading to the cache provider; the annotation alone does not promise a cluster-wide lock. With an instance-local provider such as Caffeine, 50 instances can still perform up to 50 simultaneous loads. A distributed provider can have different locking guarantees, so check its documentation before adding another lock. If the actual load count is unacceptable, serve stale values while one instance refreshes or use a short-lease distributed lock. The loader must still be safe to run twice, because a lock can expire mid-load. See [Spring synchronized caching](#).

Probe 3 (senior): Traffic to one key doubles during a sale and one cache node saturates. What do you do?

Expected reasoning: this is a hot key, not a stampede, and hashing does not spread one key. Options are a small near-cache with a short TTL in each application instance, replicating the value under several suffixed keys and reading a random one, or splitting the value. Accept the staleness window these introduce and set it deliberately.

Tier 4 - Deep dive

Internals: single flight in one JVM is a map of in-flight loads.

```
// Coalesces concurrent loads for one key inside ONE JVM only.
private final ConcurrentHashMap<String, CompletableFuture<Product>> inFlight =
    new ConcurrentHashMap<>();

CompletableFuture<Product> load(String id) {
    var mine = new CompletableFuture<Product>();
    var existing = inFlight.putIfAbsent(id, mine);
    if (existing != null) return existing.copy();    // isolate each caller's completion
    try {
        mine.complete(fetchAndPopulateCache(id));    // the winning caller does the work
    } catch (Throwable t) {
        mine.completeExceptionally(t);
        if (t instanceof InterruptedException) Thread.currentThread().interrupt();
        if (t instanceof Error error) throw error;
    } finally {
        inFlight.remove(id, mine);
    }
    return mine.copy();
}
```

The winning caller performs the fetch synchronously; returning a future does not move that work to an executor. Each caller receives a **defensive copy of the future**, so its cancellation, timeout or explicit completion cannot change the shared result. Cancelling a copy does not stop the fetch.

Probabilistic early refresh lets each request, with a probability that rises as expiry approaches, refresh the value before it expires, so expiry rarely arrives with a crowd waiting. Stale-while-revalidate keeps the old value for a short grace period while one caller fetches a new one.

Realistic failure: after a deploy every instance starts with a cold cache at once. Misses saturate the database connection pool, queries time out, and timeouts are not cached, so the cache never warms and the system stays down until traffic is shed. Fixes are staged traffic, pre-warming the hot keys, caching short-lived negative results and a limit on concurrent loads.

Trade-off: coalescing and stale serving trade freshness for load protection. Distributed locks add a component that can fail. Near-caches add an inconsistency window between instances.

How to verify: expire one hot key under load and count database queries for it. The target is about one per instance or fewer, not hundreds. Flush the cache in staging while load-testing and watch database connections and latency.

Q274 Two-phase commit, saga or outbox: how do you keep two systems consistent

Source basis: PostgreSQL PREPARE TRANSACTION.

Tier 1 - Foundation

In plain language: one business action sometimes has to change two separate systems, such as two services or a database and a message broker. There is no automatic guarantee that both changes happen or neither does. Three techniques address this, and they solve different problems.

Key terms:

- **Atomicity:** all of a change happens, or none of it does.
- **Coordinator and participants:** in two-phase commit (2PC), the coordinator asks each participant to prepare, then to commit or roll back.
- **Compensation:** an action that undoes the business effect of an earlier step, such as refunding a payment.
- **Saga:** a sequence of local transactions where failure triggers compensations for the steps already done.
- **Outbox:** an event stored in the same local transaction as the business data, then published separately.
- **Idempotent:** safe to run more than once with the same effect.

Small example: placing an order reserves stock in the inventory service and then charges payment in the payment service. If payment fails after stock is reserved, the reservation must be released.

Tier 2 - Core answer (30-second version)

Two-phase commit gives atomic commit across resources, but participants hold locks until the coordinator decides, so a coordinator failure blocks them. Across independent services I prefer a saga, which runs local transactions and compensates on failure, so intermediate states are visible and compensation must be retried. The outbox pattern is narrower: it makes "update my database and publish an event" atomic by recording the event in the same transaction. Delivery is at least once, so consumers must be idempotent.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why can two-phase commit block?

Expected reasoning: after a participant votes yes in the prepare phase it must be able to commit, so it keeps its locks until it learns the decision. If the coordinator crashes after prepare and before announcing the decision, participants wait. Recovery needs the coordinator's log, or an operator.

Probe 2 (applied): A saga's payment step fails after stock was reserved. What happens, and who drives it?

Expected reasoning: the saga runs the compensation, which releases the reservation. An orchestrator can drive this centrally, while in choreography services react to each other's events. The compensation must be idempotent and retried until it succeeds, and a saga that cannot finish needs a timeout, an alert and a manual path.

Probe 3 (senior): Someone says "we use an outbox, so we have exactly-once across services." Respond.

Expected reasoning: the outbox makes recording intent atomic with the local write. Publishing is a separate step that can repeat after a crash, so delivery is at least once. Consumers need deduplication, for example by event id, to get effectively-once processing. It does not give atomicity across two services' databases.

Tier 4 - Deep dive

Internals: in 2PC the coordinator sends *prepare*, each participant durably records that it can commit and votes, and the coordinator logs the decision and sends *commit* or *rollback*. In Java this is usually JTA with XA resources. In PostgreSQL, `PREPARE TRANSACTION` persists a transaction for a later `COMMIT PREPARED` or `ROLLBACK PREPARED`, which needs `max_prepared_transactions` above its default of 0.

Realistic failure: a coordinator crashes after prepare and nobody resolves the prepared transactions. Their locks stay, other transactions queue behind them and an abandoned prepared transaction can hold back vacuum, so table bloat grows. Query `pg_prepared_xacts` and make resolving them an operational procedure. For sagas, the common failure is a compensation that itself fails or cannot undo the effect (an email already sent), so design a **pivot step** and treat the effects after it as retryable forward steps.

Trade-off: 2PC coordinates atomic commit at the cost of availability and coupling to XA-capable resources. Isolation is a separate property of the participants and their concurrency-control protocol; 2PC alone does not guarantee globally serializable execution. Sagas keep services autonomous but expose intermediate states and push consistency logic into the application.

How to verify: crash the coordinator, or a service, between every pair of steps. Confirm the system completes or compensates, that each step is safe to repeat, and that a stuck saga appears on a dashboard.

Q275 Change data capture or a polling outbox publisher

Source basis: [Debezium documentation](#).

Tier 1 - Foundation

In plain language: with the outbox pattern, events wait in a database table. Something must move them to Kafka. A **polling publisher** is application code that repeatedly queries the table. **Change data capture (CDC)** instead reads the database's own change log and emits each committed change.

Key terms:

- **WAL (write-ahead log):** PostgreSQL's durable log of every change.
- **Logical decoding / replication slot:** PostgreSQL's mechanism to stream committed changes, with a slot remembering how far a consumer has read.
- **Connector:** a CDC component such as Debezium running in Kafka Connect.
- **At-least-once:** every event arrives, possibly more than once.
- **Event id:** a unique id that lets a consumer recognize a repeat.

Small example: after an order row and its outbox row commit, a job runs `SELECT ... FROM outbox WHERE published = false`, sends the rows and marks them sent. A CDC connector would instead see the outbox insert in the log and emit it.

Tier 2 - Core answer (30-second version)

Both deliver outbox rows to Kafka. Polling is simple but adds query load and latency, and needs care for ordering and duplicate publishers. CDC reads the transaction log, so it has low latency and commit order, but you operate another component, and a stalled PostgreSQL replication slot makes the database retain WAL and can fill the disk. Both are at least once, so consumers deduplicate.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why must consumers deduplicate with either approach?

Expected reasoning: a publisher can send an event and crash before recording that it did, and a connector can restart from an earlier position. The event is then delivered again. A stable event id and a processed-ids check, or an idempotent operation, makes this harmless.

Probe 2 (applied): How do you keep events for one order in order?

Expected reasoning: use the aggregate id as the Kafka message key so the events share a partition. With CDC, changes are read in commit order. With polling, several publishers running in parallel can send events from the same aggregate out of order, so partition the work by aggregate or use one publisher.

Probe 3 (senior): The CDC connector has been down for a day. What is the risk?

Expected reasoning: the replication slot keeps the WAL the connector has not yet consumed, so retained WAL grows and can exhaust disk. Monitor slot lag, alert on it, drop slots nobody uses, and consider `max_slot_wal_keep_size` to cap retention, knowing an invalidated slot means the connector needs a fresh snapshot.

Tier 4 - Deep dive

Internals: Debezium uses PostgreSQL logical decoding through an output plugin (commonly `pgoutput`) and a publication. Its outbox event router transformation can turn outbox rows into Kafka messages keyed by aggregate id. A common design inserts the outbox row and lets CDC pick it up from the log, so the table need not be retained for long.

Realistic failure: a polling publisher that reads `WHERE id > :last` can skip events. Transaction A takes id 100 and commits late, transaction B takes id 101 and commits first, the poller advances past 101 and never sees 100. Use a published flag, or read with a visibility-safe approach, rather than trusting id order. For CDC, the failure is the slot growth described above.

Trade-off: polling is easy to understand and operate but costs queries and delay. CDC gives timeliness and ordering but adds infrastructure, connector upgrades, schema-change handling and snapshot procedures.

How to verify: kill the publisher or connector after sending and before acknowledging, and confirm the event is delivered again but processed once. Stop the connector for an hour in a test environment and watch slot lag and disk usage.

Q276 Why can't you order events by wall-clock time across machines

Source basis: [How to do distributed locking \(fencing tokens\)](#).

Tier 1 - Foundation

In plain language: every computer has its own clock, and they are never perfectly in step. If you rely on "which timestamp is later" to decide which event happened later on different machines, you can get the wrong answer.

Key terms:

- **Wall-clock time:** the calendar time of a machine, such as `System.currentTimeMillis()`.
- **Monotonic clock:** a clock that only moves forward, suited to measuring durations, such as `System.nanoTime()`.
- **NTP:** the protocol that adjusts machine clocks, sometimes by stepping them.
- **Clock skew:** the difference between two machines' clocks.

- **Logical clock:** a counter that orders events without using real time.
- **Fencing token:** an increasing number issued with each lock or lease, checked by the resource being protected.

Small example: two servers update a user profile. Server B's clock is two seconds behind. Its later update gets an earlier timestamp, so "last write wins" discards it.

Tier 2 - Core answer (30-second version)

Machine clocks drift and get corrected, so timestamps from different machines cannot reliably order events, and last-write-wins by timestamp can silently lose a newer write. Use `nanoTime` for durations within a process and a logical mechanism for ordering, such as a serialized per-aggregate version, Lamport clocks, version vectors or hybrid logical clocks. A database sequence orders number allocation, not transaction commits; use a commit-log position or serialize the relevant writes when commit order matters. For leases and locks, add a fencing token so a stale holder cannot write.

Tier 3 - Follow-ups

Probe 1 (warm-up): When do you use `nanoTime` and when `currentTimeMillis`?

Expected reasoning: `nanoTime` for elapsed time, timeouts and measurements, because it is monotonic and unaffected by clock adjustments. `currentTimeMillis` when you need a calendar time to show or store, knowing it can jump.

Probe 2 (applied): What can a Lamport clock tell you, and what can it not?

Expected reasoning: it gives an order consistent with causality: if event A caused event B, A's number is smaller. It cannot tell whether two events with different numbers were causally related or concurrent. Version vectors can detect concurrent writes, at the cost of metadata that grows with the number of writers.

Probe 3 (senior): A client holds a lock lease, pauses for a long garbage collection, and the lease expires. What goes wrong and how do you prevent it?

Expected reasoning: after the pause the client still believes it holds the lock and writes while a new holder is active. A fencing token solves this. The lock service issues an increasing token, the client sends it with each write, and the resource rejects any token lower than one it has already seen.

Tier 4 - Deep dive

Internals: NTP can slew a clock gradually or step it, and a step can move wall time backwards. Hybrid logical clocks keep a physical component close to real time plus a counter, so timestamps stay ordered and readable. Systems such as ZooKeeper (transaction ids) and etcd (revisions) provide monotonically increasing numbers that can serve as fencing tokens. Google Spanner exposes bounded clock uncertainty with TrueTime and waits out the uncertainty when committing, which needs hardware most teams do not have.

Realistic failure: a Redis-based lock with an expiry protects a nightly job. A pause makes two workers run it at once, and both update the same rows. The lock was correct as a mutual-exclusion hint but not safe as the only protection, because nothing at the data layer checked who held it.

Trade-off: logical clocks, aggregate versions and commit-log positions provide different ordering guarantees and need coordination or extra metadata. A sequence alone is not commit order: transaction A can allocate 100 and commit after transaction B allocates 101. PostgreSQL sequence allocation is not rolled back with the transaction; see [sequence semantics](#). Wall-clock timestamps are cheap and human-readable but unreliable for ordering. Consensus-based services give strong guarantees at the price of latency and availability constraints.

How to verify: inject a `Clock` and move it backwards in tests, and confirm ordering and expiry logic do not break. Pause a lock holder past its lease in an integration test and verify the resource rejects its late write.

Q277 What is backpressure and what happens without it

Tier 1 - Foundation

In plain language: backpressure is a way for a slow part of a system to tell a fast part to slow down, so the amount of work waiting stays bounded. Without it, work piles up until the system becomes slow or runs out of memory.

Key terms:

- **Producer / consumer:** the side that creates work and the side that processes it.
- **Bounded / unbounded queue:** a queue with a maximum size, or none.
- **Load shedding:** rejecting some work on purpose to protect the rest.
- **Rejection policy:** what an executor does when it cannot accept more work.
- **Little's law:** items in a system = arrival rate x average time each spends in it.

Small example: an API accepts uploads and hands each to an executor backed by an unbounded queue. The downstream storage service slows down, the queue grows, memory rises, and every request waits longer until the service fails.

Tier 2 - Core answer (30-second version)

Backpressure keeps work in flight bounded by letting the consumer slow the producer. Use bounded queues and decide what happens when they fill: block, reject, run on the caller, or drop low-value work. Apply it at each boundary: reactive streams use `request(n)`, HTTP services return 429 or 503 with `Retry-After`, and Kafka consumers limit and pause polling. I would test it by driving the service at twice its capacity.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why is `Executors.newFixedThreadPool` risky?

Expected reasoning: it uses an unbounded `LinkedBlockingQueue`. When producers outpace the threads, the queue grows without limit, so memory and waiting time grow and nothing pushes back on the caller. Use a `ThreadPoolExecutor` with a bounded queue and an explicit rejection policy.

Probe 2 (applied): How does `CallerRunsPolicy` create backpressure, and what is its risk?

Expected reasoning: when the queue is full, the submitting thread runs the task itself, so it cannot submit more until it finishes, which naturally slows the producer. The risk is that the submitting thread may be a request thread or an event-loop thread, and running slow work there can block unrelated requests.

Probe 3 (senior): How would a Kafka consumer apply backpressure?

Expected reasoning: bound the records per poll with `max.poll.records`, and when downstream is saturated, pause assigned partitions and keep calling `poll()` so the consumer stays in the group. Stopping polling for longer than `max.poll.interval.ms` makes the broker consider the consumer failed and triggers a rebalance.

Tier 4 - Deep dive

Internals: a `ThreadPoolExecutor` adds threads beyond the core size only when its queue is full, so with an unbounded queue the maximum pool size is never reached (see Q148). Little's law makes queues measurable: at 100 requests per second and 0.5 seconds of processing, about 50 requests are in flight, and a queue holding 10,000 requests means a new arrival waits roughly 100 seconds. Reactive Streams makes demand explicit: a subscriber calls `request(n)` and the publisher must not send more than requested.

Realistic failure: a service sheds load with 503s and clients retry immediately, so the retry traffic keeps the service overloaded. Return `Retry-After`, use exponential backoff with jitter on clients, and cap retries (see Q281).

Trade-off: blocking the producer preserves work but can stall upstream threads, rejecting is fast but visible to clients, dropping loses data, and large buffers hide the problem until they fail. Choose per flow: payments should not be dropped, telemetry may be.

How to verify: drive the service at twice capacity. A healthy design shows stable latency for accepted requests and quick rejections for the rest. An unhealthy one shows growing latency and memory, then failure.

Q278 Design retries and a dead-letter topic for a Kafka consumer

Source basis: [Spring for Apache Kafka: non-blocking retries](#).

Tier 1 - Foundation

In plain language: a Kafka consumer reads messages from a partition in order. Sometimes processing a message fails. You need a plan that retries when it might help, stops when it will not, and keeps failed messages somewhere you can inspect them.

Key terms:

- **Transient failure:** one that may succeed later, such as a dependency being briefly down.
- **Permanent failure:** one that will never succeed, such as a malformed message.
- **Poison pill:** a message that fails every time and blocks progress if handled badly.
- **Offset commit:** recording how far the consumer has read.
- **Backoff:** a growing delay between attempts.
- **Dead-letter topic (DLT):** a topic holding messages that exhausted their retries.
- **Idempotent consumer:** processing the same message twice has the same effect as once.

Small example: an order event arrives with invalid JSON. Retrying a thousand times will not fix it, and if the consumer keeps retrying it, every later message on that partition waits.

Tier 2 - Core answer (30-second version)

Separate transient from permanent failures. Retry transient ones with backoff, and send permanent ones straight to a dead-letter topic with the error and original coordinates. Blocking retry keeps order while retrying but stalls the partition; moving a failed record to a DLT and replaying it later can still reorder effects. Non-blocking retry topics keep the partition moving but give up ordering for a key. Wrap deserializers so bad bytes do not become a poison pill, alert on dead-letter depth, and make consumers idempotent because retries and replays redeliver.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why is "retry forever" a bad policy?

Expected reasoning: a permanent failure never succeeds, and with blocking retry it holds up the whole partition, so lag grows for healthy messages. Bound the attempts, then divert the message to a DLT.

Probe 2 (applied): What do you give up with non-blocking retry topics?

Expected reasoning: a failed message is republished to a retry topic and the main partition continues, so a later message for the same key can be processed before the earlier one finishes retrying. If strict order per key matters, block or quarantine that key until the failure is resolved; bounded blocking retries followed by DLT

diversion do not preserve order on later replay.

Probe 3 (senior): The message cannot even be deserialized. Why does the normal error handler not help, and what do you do?

Expected reasoning: deserialization fails inside the consumer before your listener or its error handler runs, so the same record is fetched again each time. Wrap the deserializer with `ErrorHandlingDeserializer`, so the failure becomes a recoverable error that the handler can route to the DLT.

Tier 4 - Deep dive

Internals: Spring Kafka's `DefaultErrorHandler` with a back-off retries on the consumer thread, and a `DeadLetterPublishingRecoverer` publishes the exhausted record to a DLT with headers describing the exception and the original topic, partition and offset. `@RetryableTopic` creates retry topics (for example `orders-retry-0`) with delays, plus a DLT. Offset advancement depends on the container acknowledgement mode and recovery settings. Configure the recoverer to propagate DLT send failures (for example, `setFailIfSendResultIsError(true)`) and verify that unsuccessful recovery does not advance past the record. A successful asynchronous send submission alone is not proof of durable publication. See [Spring Kafka recovery and offset handling](#).

Realistic failure: the DLT fills for weeks because nobody owns it. A replay after the fix then pushes thousands of old messages through at once and overloads a dependency. Give the DLT an owner, an alert on depth and age, a documented replay tool with rate limiting, and a rule for discarding records.

Trade-off: blocking retry protects order until a record is skipped or diverted, and delays everything behind it. Non-blocking retry maximizes throughput and loses per-key order. Sending to a DLT early protects the stream but moves the problem into an operational queue.

How to verify: publish a malformed record, a record that fails twice and then succeeds, and a record that always fails. Confirm each ends up where intended, nothing is lost, and DLT depth triggers an alert. Fail the DLT send and check that the source record remains recoverable; replay an earlier record after a later one and verify the ordering policy.

APIs and delivery

Q279 How do you version and evolve an API without breaking clients

Source basis: RFC 8594: The Sunset HTTP header.

Tier 1 - Foundation

In plain language: an API is a promise to its callers about requests, responses and behavior. Changing it can break callers you do not control. Evolving an API well means making most changes in a way that existing clients keep working, and having a plan for the rare change that cannot.

Key terms:

- **Breaking change:** a change that makes a correctly written existing client fail.
- **Tolerant reader:** a client that ignores fields and values it does not recognize.
- **Deprecation:** announcing that something will be removed.
- **Sunset:** the date after which it stops working, which RFC 8594 lets a server signal in a header.
- **Contract test:** a test that checks the API still satisfies what its consumers rely on.

Small example: renaming the response field `phone` to `phoneNumber` breaks every client that reads `phone`. Adding a new optional field `phoneVerified` does not.

Tier 2 - Core answer (30-second version)

Prefer compatible evolution: adding optional fields and endpoints is safe for tolerant readers, while removing, renaming or changing meaning is breaking. When a break is unavoidable, version explicitly in the URI, a header or the media type, run both versions for a published period, and signal retirement with Deprecation and Sunset headers. Measure who still calls the old version, and use consumer-driven contract tests to catch breaks before release.

Tier 3 - Follow-ups

Probe 1 (warm-up): Is adding a value to a response enum a non-breaking change?

Expected reasoning: not automatically. A client with a strict switch may fail on an unknown value. It is only safe if the contract says clients must tolerate unknown values, so state that rule from the first version.

Probe 2 (applied): Compare URI versioning with header versioning.

Expected reasoning: URI versions are visible, easy to route, cache and test from a browser. Header or media-type versions keep resource URIs stable and match the idea that the URI identifies the resource, but are harder to see, test and cache correctly. Choose one convention for the whole organization.

Probe 3 (senior): How do you retire v1 safely?

Expected reasoning: announce a date, send deprecation and sunset signals, identify remaining callers from request metadata such as a client id header, contact them, reduce traffic gradually and then remove it. Do not remove on schedule if important consumers remain, unless the policy says so and that was communicated.

Tier 4 - Deep dive

Internals: a common structure keeps one domain model and maps it to separate request and response models per API version, so version differences stay at the edge. Versioning the whole API gives clients one number to track, while versioning single resources limits churn but is harder to reason about. Recent Spring versions add built-in API versioning support, so check whether yours does rather than assuming. Event schemas need the same discipline, and a schema registry can enforce backward, forward or full compatibility modes between producer and consumer versions.

Realistic failure: a team changes amount from cents to dollars without changing the name. No schema checks fail, no client crashes, and totals are silently wrong. Changes in meaning are the most dangerous breaks because tooling does not see them. Add a new field with a new name, deprecate the old one, and document units in the contract.

Trade-off: every supported version is more code, tests and operational burden, but removing versions too quickly damages trust and forces clients to upgrade in lockstep with you.

How to verify: run the previous release's client test suite or contract tests against the new server in CI. Add a field and confirm old clients keep working. Track per-version call counts before retirement.

Q280 What are ETags and conditional requests used for

Source basis: RFC 9110: Conditional requests.

Tier 1 - Foundation

In plain language: an ETag is a label the server attaches to a specific version of a resource, such as a fingerprint. The client can send it back to ask, "has it changed?" or to say, "only update it if it has not changed." That makes caching cheaper and prevents one user from silently overwriting another's edit.

Key terms:

- **ETag:** an opaque validator for a representation of a resource.
- **If-None-Match:** the client's "send it only if it differs from this ETag."
- **If-Match:** the client's "apply this change only if the resource still has this ETag."
- **304 Not Modified:** the response when the cached copy is still current.
- **412 Precondition Failed:** the response when an If-Match condition is false.
- **428 Precondition Required:** the response when a conditional header is required and missing.
- **Lost update:** two writers read the same version and the later write erases the earlier one.

Small example: two admins open the same customer record. The first saves a new address. The second then saves a change to the phone number, based on the old copy. With If-Match, the second save fails with 412 instead of overwriting the address.

Tier 2 - Core answer (30-second version)

An ETag identifies a specific version of a resource. For reads, If-None-Match returns 304 with no body when nothing changed. For writes, If-Match makes the update conditional, and the server returns 412 if the resource changed since the client read it, which prevents lost updates. I would derive a strong ETag from JPA @Version only if every change to the selected representation advances that version, including any related data it contains, and distinguish representation variants. Otherwise I need a representation-wide revision or hash as well as the database concurrency check.

Tier 3 - Follow-ups

Probe 1 (warm-up): What is the difference between how 304 and 412 are used?

Expected reasoning: 304 answers a conditional read: the cached copy is still valid, so no body is sent. 412 answers a conditional write: the precondition failed, so the change was not applied.

Probe 2 (applied): How do you map this to JPA?

Expected reasoning: expose @Version as a strong ETag only when it changes for every observable change to the representation. If the response includes related entities, an independent child update may leave the root version unchanged; use an aggregate revision or representation hash and ensure the write transaction validates all relevant revisions atomically. Include a variant identifier when negotiated representations differ. For a single versioned entity, compare the client's If-Match with the current version and let the persistence layer enforce the version in UPDATE ... WHERE version = ?. An application-only comparison followed by an unguarded save leaves a race. Map a stale conditional update to 412; 409 can describe a conflict outside an HTTP precondition. See [RFC 9110 strong validators](#).

Probe 3 (senior): What does Spring's ShallowEtagHeaderFilter do, and what does it not do?

Expected reasoning: it computes an ETag from the response body and answers 304 when it matches If-None-Match, which saves bandwidth. The server still builds the full response, so it saves no processing, and it is not a concurrency control for writes.

Tier 4 - Deep dive

Internals: ETag values are quoted strings, and a weak validator is prefixed w/. RFC 9110 says If-Match uses strong comparison, while If-None-Match uses weak comparison. If-None-Match: * on a PUT can mean "create only if it does not exist." A server can require conditional headers on updates and answer 428 when they are missing.

Realistic failure: an ETag computed from a content hash that leaves out one field means changes to that field never alter the ETag, so clients are told their copy is current when it is not. Another failure is differing ETags across instances behind a load balancer because of per-node compression or ordering, which defeats caching and breaks If-Match.

Trade-off: conditional writes force clients to read before writing and handle conflicts. That is the right cost when overwrites matter, and unnecessary friction for data where last-write-wins is acceptable.

How to verify: read a resource in two clients, update it from the first, then update from the second with the old ETag. Expect 412 and an intact first update. Change a related field included in the representation without updating the root entity, and confirm the ETag still changes. Check distinct negotiated representations use valid validators. Request with a current If-None-Match and confirm 304 and no body.

Q281 How do you set timeouts and retries so one slow dependency does not take everything down

Tier 1 - Foundation

In plain language: a call to another service can be slow or never return. If callers wait forever, threads and connections fill up and the failure spreads. Timeouts limit how long you wait, and retries try again when a failure may be temporary, but careless retries make an outage worse.

Key terms:

- **Connect timeout / read timeout:** limits for establishing a connection and for waiting for data.
- **Deadline:** the total time allowed for the whole user request.
- **Backoff and jitter:** growing, randomized delays between retries.
- **Idempotent:** repeating the operation has the same effect as doing it once.
- **Retry budget:** a cap on how much traffic may be retried.
- **Circuit breaker:** a switch that stops calls to a failing dependency for a while.

Small example: service A calls B, which calls C, and both call layers allow three retries after the initial attempt. That is four attempts per layer, so one failing request can become $4 \times 4 = 16$ attempts against C, which is already struggling. With three call layers allowing three total attempts each, the multiplier would be $3 \times 3 \times 3 = 27$. Always distinguish retries from total attempts.

Tier 2 - Core answer (30-second version)

Every remote call needs a timeout, and timeouts should shrink down the call chain inside one overall deadline. Retry only safe, idempotent operations on retryable errors, with exponential backoff and jitter, at one layer, and cap retries with a budget, because retries multiply across layers. Add a circuit breaker so a clearly failing dependency fails fast, and a bulkhead so it cannot use every thread.

Tier 3 - Follow-ups

Probe 1 (warm-up): Which operations are safe to retry?

Expected reasoning: reads and operations that are idempotent by design, or made idempotent with an idempotency key. A non-idempotent POST such as "charge the card" can duplicate its effect if the first attempt actually succeeded but the response was lost.

Probe 2 (applied): Why add jitter to backoff?

Expected reasoning: without randomization, many clients that failed together retry together, producing synchronized load spikes. Jitter spreads retries over time.

Probe 3 (senior): Where do you put retries, and how do deadlines help?

Expected reasoning: retry at one layer, usually the closest to the caller that can judge whether retrying is appropriate, and avoid retries in every layer. Pass the remaining deadline downstream, as gRPC does, so a downstream call is never given more time than the caller has left and abandoned work stops.

Tier 4 - Deep dive

Internals: timeouts bound thread occupancy. With 200 worker threads and a dependency that hangs for 30 seconds, even a modest request rate exhausts the pool within seconds, and unrelated endpoints stop responding. A circuit breaker moves between closed, open and half-open states using failure rate or slow-call thresholds. A retry budget limits retries to a small fraction of normal traffic, so a widespread failure cannot be multiplied. Resilience4j's aspect order matters, as discussed in Q165.

Realistic failure: a dependency slows from 50 ms to 5 s. Clients retry, the dependency receives three times the load, and recovery is impossible until traffic is shed. The fix is shorter timeouts, one retry layer, a budget, a breaker, and a fallback that serves cached or reduced data.

Trade-off: a short timeout fails requests that would have succeeded slowly, and a long one ties up resources. Retries improve success under transient failure and worsen overload. Set timeouts from the dependency's measured latency, for example a high percentile with headroom, then revisit them.

How to verify: inject latency and errors into a dependency in staging. Confirm callers time out within budget, total attempts stay bounded, and unrelated endpoints remain healthy.

Q282 Blue-green, canary and rolling deployments: how do you choose, and what about the database

Tier 1 - Foundation

In plain language: releasing a new version without downtime means old and new code run side by side for a while. Different strategies control how traffic moves, and the database and message formats must work with both versions at once.

Key terms:

- **Rolling update:** replace instances a few at a time.
- **Blue-green:** run a full second environment and switch traffic to it.
- **Canary:** send a small share of real traffic to the new version first.
- **maxSurge / maxUnavailable:** Kubernetes settings for how many extra or missing pods are allowed during a rolling update.
- **Expand/contract:** change the schema in compatible steps, adding first and removing later.

Small example: you add a column. During the rollout, old pods that do not know the column and new pods that need it both talk to the same database.

Tier 2 - Core answer (30-second version)

Rolling updates replace instances gradually and are the Kubernetes default. Blue-green switches between two full environments and makes rollback a switch back, at the cost of double capacity. A canary exposes a small share of real traffic first and widens it only if error rate and latency stay healthy, and it needs traffic splitting. In every case old and new versions overlap, so database changes and message formats must be backward compatible, using expand/contract.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why must a database change be backward compatible during a rolling update?

Expected reasoning: for part of the rollout, old pods still run against the new schema. A change that removes or renames something the old version uses makes it fail mid-deployment.

Probe 2 (applied): What do you watch during a canary, and how do you decide?

Expected reasoning: error rate, latency percentiles, saturation and a key business metric, compared against the baseline version over enough traffic to be meaningful. Define automatic abort thresholds beforehand. One percent of a low-traffic endpoint may be too few requests to show a problem.

Probe 3 (senior): The new version is bad and the migration already ran. How do you roll back?

Expected reasoning: roll back the code, not the schema. Because the migration was an expansion that the old code tolerates, the old version works against it. Destructive "contract" steps happen only in a later release, after the new version is stable.

Tier 4 - Deep dive

Internals: a Kubernetes Deployment creates a new ReplicaSet and shifts replicas while readiness probes gate traffic, constrained by `maxSurge` and `maxUnavailable`, both of which default to 25%. Canary and blue-green traffic shifting needs an ingress controller, service mesh or a rollout controller such as Argo Rollouts or Flagger. Expand/contract in practice:

1. Expand: add the new column or table, nullable or with a default, without breaking the old version.
2. Deploy code that writes both forms atomically while continuing to read the old form. Wait until every old-only writer, including jobs, has stopped.
3. Backfill in bounded batches using row locking or version checks so a stale backfill cannot overwrite a concurrent update. Validate completeness and agreement.
4. Switch reads to the new form while maintaining both writes for the rollback window. A non-null new value is not proof of freshness while old-only writers remain.
5. After the rollback window, stop using the old form and remove it in a later contract release. See [GitLab's migration guidance](#) for staged changes and synchronization approaches.

The same applies to message formats, since old and new consumers or producers may overlap.

Realistic failure: a migration renames a column in the same release that uses the new name. Requests handled by old pods fail as soon as they access the renamed column, even if new pods become ready. Readiness alone cannot make an incompatible schema safe.

Trade-off: blue-green gives fast switching and simple rollback but needs double capacity and complicates stateful data. Canary gives the safest real-traffic validation and needs routing and observability. Rolling is simplest but exposes users to the new version gradually, with slower rollback.

How to verify: deploy a deliberately bad version to a small slice and confirm the metric gate stops it. Run old and new versions against the migrated schema in CI before releasing. Interleave an old-only write, a dual write and a backfill; verify that reads stay correct and rollback works throughout its supported window.

Q283 Why do pooled HTTP connections fail intermittently after idle periods

Tier 1 - Foundation

In plain language: opening a network connection is expensive, so HTTP clients keep connections open and reuse them. Something in the path, such as the server, a load balancer or a firewall, may close a connection that has been idle too long. The client does not notice until it tries to use it, and that request fails.

Key terms:

- **Keep-alive:** keeping a connection open for further requests.
- **Connection pool:** a set of reusable open connections.
- **Idle timeout:** how long an unused connection may stay open before something closes it.
- **Stale connection:** one the other side has already closed.
- **Connection reset:** an error shown when you use a connection the peer has dropped.
- **Max connections per route:** the pool limit for one destination.

Small example: a service calls a partner API every few minutes. The first call after a quiet period often fails with "connection reset," and the next call works.

Tier 2 - Core answer (30-second version)

Pooled connections that sit idle can be closed by the server, load balancer or NAT, and the client discovers it only when it reuses the connection, so the first request after a quiet period fails. Make the client's idle timeout shorter than the shortest idle timeout in the path, evict or validate idle connections before reuse, and retry idempotent requests once. Size request capacity from request rate times latency, accounting for HTTP/2 multiplexing, and coordinate connection lifetime with DNS cache expiry so reconnects can notice address changes.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does only the first request after idle fail?

Expected reasoning: the idle connection was closed by the other side. The failure on reuse removes it from the pool, and the next request opens a fresh connection, which works.

Probe 2 (applied): How do you fix it?

Expected reasoning: set the client's maximum idle time below the lowest idle timeout in the path (server, load balancer, firewall), evict idle connections in the background or validate a connection that has been idle before reuse, and retry once for idempotent requests. The settings and names differ between clients such as Apache HttpClient, Reactor Netty and the JDK client, so check the documentation.

Probe 3 (senior): How do you size the pool, and what do exhaustion symptoms look like?

Expected reasoning: by Little's law, in-flight requests are roughly request rate times average latency, per destination, plus headroom. With HTTP/1.1 without pipelining, that approximates occupied connections. HTTP/2 multiplexes streams on a connection, so size stream capacity and connections using client/server limits and measurements; see [RFC 9113 streams](#). Too small a pool shows as requests waiting to lease a connection and "connection request timeout" errors, even though the dependency is healthy. Too large a pool can overload the dependency.

Tier 4 - Deep dive

Internals: when a peer closes a TCP connection the client may not learn of it until it writes, and then receives a reset. A race remains even with validation, because the peer can close the connection between the client's check and its write; a bounded retry can recover when the operation is safe to repeat. An existing pooled connection keeps its original peer address. A maximum connection lifetime forces eventual replacement, but a new connection can still use a cached DNS address. For clients using the JVM resolver, `networkaddress.cache.ttl` controls positive caching and `-1` means cache forever; some clients have their own resolvers and cache settings. Coordinate connection replacement with the actual resolver's finite TTL and failover behavior. See [Java 21 DNS caching](#).

Realistic failure: a load balancer closes idle connections after 60 seconds, while the client keeps them for five minutes. Roughly one request per quiet period fails, in a pattern that looks random and appears only in production. The check is to compare the idle timeouts of every hop with the client's setting.

Trade-off: aggressive eviction and short lifetimes cost more handshakes, including TLS, and add latency and CPU. Long-lived idle connections save that cost and risk stale reuse. Retries cover the gap only for idempotent calls.

How to verify: reproduce by waiting past the idle timeout and sending one request, then confirm the fix removes the failure. In a separate failover test, change the resolved address and verify new connections use it within the combined DNS-cache and connection-lifetime policy. Expose pool metrics such as leased, available and pending connections, through Micrometer where the client supports it, and alert on pending requests.

Q284 What does a PodDisruptionBudget protect against, and what can it not do

Source basis: [Kubernetes: Specifying a Disruption Budget](#).

Tier 1 - Foundation

In plain language: Kubernetes sometimes removes pods on purpose, for example when a node is drained for an upgrade. A PodDisruptionBudget (PDB) says how many pods of an application may be unavailable at once during such planned removals, so maintenance does not take the whole service down.

Key terms:

- **Voluntary disruption:** a planned removal such as a node drain or an eviction.
- **Involuntary disruption:** an unplanned loss such as a node crash or an out-of-memory kill.
- **Eviction:** the API request to remove a pod gracefully.
- `minAvailable` / `maxUnavailable`: the budget expressed as pods that must stay, or may go.
- **Topology spread:** a scheduling rule that places replicas across nodes or zones.

Small example: an app has three replicas and a PDB with `minAvailable: 2`. Draining a node can evict one pod at a time, and the drain waits if another is already unavailable.

Tier 2 - Core answer (30-second version)

A PodDisruptionBudget limits how many pods can be down during voluntary disruptions like node drains, by making the eviction API refuse an eviction that would break the budget. It does not protect against involuntary failures such as a node crash or an out-of-memory kill, and it does not add capacity. Use it with enough replicas spread across nodes and zones, and avoid budgets that cannot be satisfied, because they block maintenance.

Tier 3 - Follow-ups

Probe 1 (warm-up): Does a PDB stop an out-of-memory kill?

Expected reasoning: no. A PDB governs voluntary evictions. A crash, an OOM kill or a node failure is involuntary, so it still happens regardless of the budget.

Probe 2 (applied): One replica and `minAvailable: 1`. What happens during a node drain?

Expected reasoning: the eviction would take availability below the budget, so it is refused and the drain stalls. The same occurs when `minAvailable` equals the replica count. Either run more replicas, or use `maxUnavailable` and accept a short outage deliberately.

Probe 3 (senior): How do you make sure replicas survive a single node or zone loss?

Expected reasoning: a PDB is not enough, because it does not decide placement. Use topology spread constraints or pod anti-affinity to place replicas on different nodes and zones, run enough replicas, and combine with readiness probes and graceful shutdown.

Tier 4 - Deep dive

Internals: when an eviction would violate a budget the API server rejects it with a 429 response, and tools such as `kubectl drain` retry. A direct `kubectl delete pod` is not an eviction and bypasses the budget. Cluster autoscaler scale-down and managed-platform node upgrades generally respect PDBs, though managed upgrades may stop honoring a budget after a time limit, so check your provider's documentation.

Realistic failure: a team sets `minAvailable: 100%` on every service. A cluster upgrade then stalls for hours because no pod can ever be evicted, and an engineer force-deletes pods to finish, losing the protection the budget was meant to give.

Trade-off: a strict budget protects availability and slows or blocks maintenance. A loose one speeds upgrades and risks more simultaneous loss. Pick the budget from the service's real tolerance and replica count.

How to verify: run `kubectl get pdb` and read the allowed-disruptions value. Drain a node in a test cluster and confirm evictions wait rather than dropping below the budget. Create an unsatisfiable PDB and observe the stuck drain, so the symptom is recognizable.

Q285 Service mesh or library-based resilience

Tier 1 - Foundation

In plain language: a service mesh puts a small proxy next to each service so the platform handles network concerns such as encryption, retries and traffic routing. A resilience library does similar work inside the application code. The choice is about where the logic lives and who controls it.

Key terms:

- **Sidecar:** a proxy container deployed alongside each application container.
- **Data plane / control plane:** the proxies that carry traffic, and the component that configures them.

- **mTLS:** mutual TLS, where both sides authenticate with certificates.
- **Traffic splitting:** sending a percentage of requests to different versions.
- **Fallback:** an alternative response when a call fails.

Small example: a mesh retries a failed call to a backend automatically, with no code change. A library lets the code, on failure, return the last cached price instead.

Tier 2 - Core answer (30-second version)

A mesh moves cross-cutting network behavior, such as mTLS, retries, timeouts, traffic splitting and telemetry, into the platform and works across languages. A library runs in the process, so it can use business context, such as returning cached data or failing one operation without affecting another. A mesh adds latency, resources and operational complexity, and the biggest trap is stacked retries from the mesh, the client and the application. Choose one owner for each behavior.

Tier 3 - Follow-ups

Probe 1 (warm-up): What can a mesh give you that a library cannot?

Expected reasoning: uniform behavior across languages without code changes, automatic mTLS between services with certificate rotation, consistent telemetry, and platform-controlled traffic management such as canary routing.

Probe 2 (applied): What can a library do that a mesh cannot?

Expected reasoning: decisions that depend on application meaning. A mesh sees requests and responses, not business context, so it cannot choose a sensible fallback value or treat two operations in the same service differently based on domain rules.

Probe 3 (senior): How do you prevent retry amplification?

Expected reasoning: list every layer that can retry, including mesh, client library and application, and compute the worst case by multiplying. Assign retries to one layer, restrict retry conditions to idempotent requests and specific failures, and add a retry budget.

Tier 4 - Deep dive

Internals: proxies, commonly based on Envoy, intercept traffic and are configured by the control plane. Certificates are issued and rotated automatically for mTLS. Each hop adds some latency and resource use, which you should measure for your workload. Some meshes offer modes without a per-pod sidecar, and their maturity and limits vary by version, so check yours.

Realistic failure: a mesh policy retries on any 5xx, including for a non-idempotent POST. A timeout on a payment endpoint then triggers a retry, and the customer is charged twice. Restrict retry rules to idempotent methods or require idempotency keys. Debugging also gets harder: a 503 may come from the proxy, not the application, so learn the proxy's response flags and read its logs.

Trade-off: a mesh gives consistent policy and security at scale in return for platform complexity and a team to run it. Library resilience is simple for a few services in one language and drifts across teams in a larger fleet.

How to verify: inject failures and watch whether traffic to the failing service multiplies. Check which layer made each retry. Test that mTLS is enforced by attempting a plaintext call.

JVM and Java

Q286 How do you choose a garbage collector

Source basis: [JEP 439: Generational ZGC](#).

Tier 1 - Foundation

In plain language: the JVM automatically frees objects that nothing can reach any more. Different collectors make different trade-offs between how long the application pauses, how much work it completes, and how much memory the collector itself needs.

Key terms:

- **Heap:** the memory where Java objects live.
- **Young / old generation:** new objects versus long-lived ones, collected at different frequencies.
- **Stop-the-world pause:** a moment when application threads are halted for collection work.
- **Throughput:** the share of time spent running the application rather than collecting.
- **Allocation rate:** how fast the application creates objects.

Small example: a payment API has an acceptable average latency but its 99th-percentile latency spikes every few minutes, and the spikes line up with garbage collection pauses.

Tier 2 - Core answer (30-second version)

Start from the goal. G1 is the default on modern JDKs and a good general choice, with a pause-time target it tries to meet. ZGC targets very low pauses that do not grow with heap size, at some cost in throughput and memory. Parallel GC maximizes throughput for batch work, and Serial suits tiny heaps. Confirm which collector is actually running, tune from GC logs and load tests, and often fix allocation behavior before changing collectors.

Tier 3 - Follow-ups

Probe 1 (warm-up): What does `-XX:MaxGCPauseMillis` do?

Expected reasoning: it sets a pause-time goal for G1, 200 ms by default. G1 adjusts its work to try to meet it, but it is a target, not a guarantee, and a very low target can reduce throughput.

Probe 2 (applied): Your container runs a different collector than you expected. Why?

Expected reasoning: the JVM selects a collector from the resources it detects. In a container with very few CPUs or little memory it can choose Serial instead of G1. Check with `-Xlog:gc` or `-XX:+PrintFlagsFinal`, and set the collector explicitly if it matters.

Probe 3 (senior): When would you move from G1 to ZGC?

Expected reasoning: when measured pauses violate a latency objective, especially with large heaps, and the throughput and memory cost is acceptable. In JDK 21 generational mode is enabled with `-XX:+UseZGC -XX:+ZGenerational`, and later releases changed the defaults, so check your JDK. Compare both under realistic

load before committing.

Tier 4 - Deep dive

Internals: G1 splits the heap into regions and collects the ones with most garbage first, using young and mixed collections. Very large objects (humongous allocations, see Q205) are handled specially and can cause trouble. If evacuation cannot find space it falls back to a long full collection. ZGC does most work concurrently using load barriers and concurrent relocation, so pause times stay short regardless of heap size, with extra memory and CPU overhead.

Realistic failure: a team raises the heap to stop long pauses, and the container is killed by the kubelet for exceeding its memory limit, because the heap is only part of the JVM's total memory (it also uses metaspace, thread stacks and native memory). The real cause was a high allocation rate from a hot loop.

Trade-off: lower pauses generally cost throughput and memory, higher throughput generally costs longer pauses. Tuning flags rarely beats reducing allocation or sizing the heap and container correctly.

How to verify: enable `-Xlog:gc*` and examine pause distribution, allocation rate and old-generation behavior. Run the same load with G1 and ZGC and compare p99 latency, throughput and memory.

Q287 How do you find a memory leak in a production JVM

Source basis: [JDK 21 jcmd diagnostic impact](#).

Tier 1 - Foundation

In plain language: in Java a leak is not lost memory. It is objects that are still reachable, so the collector cannot free them, but the program no longer needs them. Memory use keeps growing until the process fails.

Key terms:

- **GC root:** a starting reference the collector trusts, such as a static field or a thread stack.
- **Reachable:** an object connected to a GC root through references.
- **Heap dump:** a snapshot of all objects in the heap.
- **Retained size:** the memory that would be freed if an object were removed.
- **Dominator tree:** a view showing which objects keep the most memory alive.
- **Native Memory Tracking (NMT):** JVM tooling to see non-heap memory use.

Small example: code stores each session in a static Map keyed by session id and never removes finished sessions, so the map grows with every user.

Tier 2 - Core answer (30-second version)

First establish which memory is growing: Java heap, metaspace, native memory, or the container limit being hit. A heap leak shows old-generation usage that keeps rising after full collections. Start with existing metrics and JFR evidence; take a histogram or heap dump only after assessing its impact, then use a tool like Eclipse MAT to find the dominator and follow the path to the GC root. Typical causes are static collections, unbounded caches, forgotten listeners, ThreadLocals in pooled threads and class-loader leaks.

Tier 3 - Follow-ups

Probe 1 (warm-up): How do you tell a leak from a heap that is simply too small?

Expected reasoning: look at memory after full collections over time. A leak shows a rising floor that never comes back down. An undersized heap shows a stable floor that is close to the limit, with frequent collections.

Probe 2 (applied): The pod restarts with no Java exception. Is it a heap leak?

Expected reasoning: not necessarily. If Kubernetes reports OOMKilled, the container exceeded its memory limit, and the cause may be native memory, direct buffers, thread stacks or metaspace, not the Java heap. Check NMT and the container's memory metrics, not only heap usage.

Probe 3 (senior): What are the risks of taking a heap dump from production?

Expected reasoning: it can pause the application, needs disk space comparable to the heap, and contains data such as personal information and secrets. Take it from one instance that has been removed from rotation if possible, store and delete it under the data policy, and start with metrics or JFR sampling where they can answer the question. A histogram produces less output than a dump but is also documented as high impact; do not treat it as a cheap, pause-free first step.

Tier 4 - Deep dive

Internals: useful commands:

```
jcmd <pid> GC.class_histogram | head -30      # HIGH impact; head only trims output
jcmd <pid> GC.heap_dump /tmp/heap.hprof      # pauses the app; check disk space
jcmd <pid> JFR.start settings=profile duration=5m filename=/tmp/rec.jfr
# Native memory needs the JVM started with -XX:NativeMemoryTracking=summary:
jcmd <pid> VM.native_memory summary
```

Taking two histograms some time apart shows which classes grow, but each scan can impose a substantial pause or trigger collection depending on options and collector. The JDK classifies both `GC.class_histogram` and `GC.heap_dump` as high impact. Limit production use to an assessed diagnostic window; truncating output with `head` does not reduce heap-inspection work. In MAT, the dominator tree and retained size point at the structure holding the memory, and the path to GC roots shows who holds the reference. JFR's old-object sampling can show where leaking objects were allocated with less overhead than a dump. Starting the JVM with `-XX:+HeapDumpOnOutOfMemoryError` captures evidence at failure.

Realistic failure: a pooled thread stores a request context in a `ThreadLocal` and never clears it. Because the pool reuses threads, each holds its last context forever, and a large object graph stays reachable. The fix is to clear the value in a `finally` block or avoid the `ThreadLocal`.

Trade-off: diagnostic data costs pause time, disk and privacy risk, while waiting for a crash costs availability. Gather the cheapest evidence that can answer the question first.

How to verify: reproduce growth in a soak test, apply the fix and confirm that the post-collection floor levels off.

Q288 What are the trade-offs of GraalVM native images for Spring Boot

Source basis: [Spring Boot native images](#).

Tier 1 - Foundation

In plain language: a normal Java application starts a virtual machine, loads classes and compiles hot code while running. A native image is compiled ahead of time into a standalone executable, so it starts faster and needs less memory, in exchange for restrictions on dynamic features.

Key terms:

- **AOT (ahead-of-time) compilation:** compiling before the program runs.
- **JIT (just-in-time) compilation:** compiling hot code while the program runs.

- **Closed-world assumption:** the build must know every class that can be reached.
- **Reflection hints:** build-time metadata telling the compiler which classes are used reflectively.
- **Warm-up:** the time a JVM needs to reach full speed.

Small example: a function that must scale from zero starts in milliseconds as a native image, instead of seconds as a JVM application.

Tier 2 - Core answer (30-second version)

A native image gives much faster startup and lower idle memory, which suits serverless, scale-to-zero and dense deployments. The cost is the closed-world assumption: reflection, proxies and resources must be known at build time, and Spring Boot's AOT processing generates many of the hints but libraries can need more. Builds are slower and heavier, tooling differs, and peak throughput after warm-up can be lower than a tuned JVM. I would choose it only when startup time or footprint is a real constraint, and test the native build itself.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does reflection cause problems?

Expected reasoning: the build analyzes reachable code and removes the rest, and reflection finds classes by name at run time, which the analysis cannot see unless told. Missing hints produce runtime failures that never occurred on the JVM.

Probe 2 (applied): It works on the JVM and fails in the native image. How do you find the cause?

Expected reasoning: look for missing reflection, proxy, resource or serialization hints. Run the tests against the native build, use the GraalVM tracing agent to find what the application uses dynamically, and add hints through Spring's runtime hints API or registration annotations.

Probe 3 (senior): When is the JVM the better choice?

Expected reasoning: for long-running services with stable load, where warm-up is amortized and peak throughput, mature tooling and simple builds matter more than startup. Compare measured startup, memory and steady-state latency, and consider alternatives that keep the JVM, such as lazy initialization, trimmed auto-configuration and class-data sharing.

Tier 4 - Deep dive

Internals: the native-image builder performs a static analysis from the entry points, includes only reachable code, initializes some classes at build time and produces an executable with its own runtime and garbage collector. Spring Boot's AOT step generates bean definitions and hints ahead of time, so the application does

less discovery at startup. Features that rely on runtime class generation or loading behave differently, and some JVM tooling such as standard JFR and heap dump workflows differs or is unavailable, so check what your GraalVM distribution supports.

Realistic failure: a DTO serialized by Jackson works in all JVM tests. In the native build the JSON comes back empty or fails because no hint registered its members for reflection. It reaches production because only JVM tests ran. Run the integration tests in native mode in CI.

Trade-off: startup and memory gains come with longer builds, a build that can need several gigabytes of memory, less flexibility with dynamic libraries and possibly lower peak throughput. The operational question is whether the gain changes your cost or scaling behavior.

How to verify: build both forms, run the same test suite against each, and measure startup time, resident memory and steady-state latency under realistic load.

Q289 What are structured concurrency and scoped values (a later-version comparison)

Source basis: [Java 21 StructuredTaskScope API](#) and [Java 21 ScopedValue API](#).

Label: in Java 21 both are **preview** APIs requiring `--enable-preview`, and their status and API shape have changed in later releases. This question is a comparison with the guide's Java 21 baseline. Check the final status in your JDK before relying on either.

Tier 1 - Foundation

In plain language: structured concurrency treats several related tasks as one unit of work that starts together and finishes together, like a block of code. Scoped values let code share a read-only binding with everything called inside a bounded region, as a safer alternative to thread-local variables. The bound object itself can still be mutable.

Key terms:

- **Structured concurrency:** subtasks cannot outlive the scope that started them.
- **StructuredTaskScope:** the Java 21 preview class that groups subtasks.
- **Fork / join:** start a subtask, then wait for the group.
- **Scoped value:** a read-only binding for the duration of a scope; it does not freeze the bound object.
- **ThreadLocal:** a mutable per-thread variable, with a lifetime tied to the thread.

Small example: loading a page needs a user and their orders. Both are fetched concurrently, and if one fails the other should be cancelled and the failure reported.

Tier 2 - Core answer (30-second version)

Structured concurrency groups related tasks so they cannot outlive the scope, errors propagate and a failure can cancel the siblings, which avoids leaked threads and orphaned futures. Scoped values share read-only bindings, preferably to immutable data such as a request id, with code called within a bounded scope, and are meant as a safer alternative to ThreadLocal, especially with many virtual threads. In Java 21 both are preview, so I would not ship them to production without a deliberate decision.

Tier 3 - Follow-ups

Probe 1 (warm-up): What problem does structured concurrency solve compared with hand-built futures?

Expected reasoning: with ad hoc futures, a failed task may leave others running, an exception can be lost, and no code block owns the lifetime. A scope gives the tasks a clear owner and a defined end.

Probe 2 (applied): How does `ShutdownOnFailure` behave when one subtask throws?

Expected reasoning: the first failing subtask shuts down the scope and interrupts unfinished siblings. `join()` waits for all subtasks to finish or the scope to shut down, so it can return before an interrupted sibling stops. `throwIfFailed()` reports the first failure. The implicit `close()` at the end of the try-with-resources block waits for remaining threads to terminate; a sibling that ignores interruption can block closure indefinitely. Cancellation is cooperative: use interruptible work and bounded dependency calls, and do not treat `joinUntil()` as a hard bound on scope closure.

Probe 3 (senior): Why prefer a scoped value to a `ThreadLocal` here?

Expected reasoning: a `ThreadLocal` is mutable, lives as long as its thread, and with pooled threads can leak data between requests. Copying many thread-locals into many virtual threads also costs memory. A scoped-value binding is read-only within its scope, can be rebound in a nested scope, and is inherited by subtasks forked in a structured scope. The referred-to object is not automatically immutable; use immutable objects or synchronize shared mutation.

Tier 4 - Deep dive

Internals:

```
// Java 21 only with --enable-preview; the API may differ in later releases.
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var user    = scope.fork(() -> loadUser(id));
    var orders = scope.fork(() -> loadOrders(id));
    scope.join().throwIfFailed();           // failure triggers sibling interruption
    return new Page(user.get(), orders.get());
} // close waits for unfinished subtasks to terminate
```

By default each fork runs in a new virtual thread. `ShutdownOnSuccess` is the counterpart for "first result wins." A scoped value is bound with `ScopedValue.where(...)` and read within that call tree. Neither mechanism replaces a transaction or a dependency timeout. `joinUntil()` bounds that join, but `close()` still waits for unfinished subtasks; their work must respond to interruption or have its own timeout.

Realistic failure: a team uses a preview API in a library, upgrades the JDK, and the API changes shape or the preview flag is no longer accepted, so the build breaks. Preview features are for evaluation, and production use needs an owner who tracks the JEPs.

Trade-off: the structured form is easier to reason about and clean up. In return it requires newer APIs and, in Java 21, preview flags. `CompletableFuture` works everywhere and is flexible, and cancellation and lifetime handling are manual.

How to verify: make one subtask throw and confirm the sibling receives interruption. Hold that sibling behind a test latch and verify `join()` can return while it is unfinished; release it and verify `close()` waits for its termination. Confirm the scoped binding disappears outside its scope, a nested rebind restores the outer binding, and a bound mutable object remains mutable. Use immutable objects or synchronization before sharing it with subtasks.

Q290 ClassNotFoundException versus NoClassDefFoundError, and how class loading causes dependency bugs

Tier 1 - Foundation

In plain language: Java loads classes on demand from the classpath. If a class is missing, or two versions of a library are present, the program fails at run time rather than at compile time. The two error names look similar but describe different situations.

Key terms:

- **Classpath:** where the JVM looks for classes.
- **Class loader:** the component that finds and loads a class.
- **Parent delegation:** a loader asks its parent before looking itself.
- **Transitive dependency:** a library your dependency depends on.
- **ServiceLoader:** the Java mechanism for discovering implementations of an interface.

Small example: your code compiles against library version 2, but another dependency pulls in version 1, so at run time a method that exists in 2 is missing.

Tier 2 - Core answer (30-second version)

`ClassNotFoundException` happens when code asks to load a class by name and it is not found. `NoClassDefFoundError` means a class that was there at compile time is missing at run time, or failed to initialize. Most real incidents in a Maven project come from version conflicts between transitive dependencies, which show up as `NoSuchMethodError` or `LinkageError`. Diagnose with `mvn dependency:tree`, then pin or exclude versions, and use the Enforcer plugin to catch conflicts at build time.

Tier 3 - Follow-ups

Probe 1 (warm-up): What is the difference between the two errors?

Expected reasoning: `ClassNotFoundException` is a checked exception from an explicit load such as `Class.forName` or reflection. `NoClassDefFoundError` is an error raised when the JVM, while linking or running code that was compiled against a class, cannot find or initialize it.

Probe 2 (applied): A `NoClassDefFoundError` appears for a class that is on the classpath. Why?

Expected reasoning: its static initializer probably failed earlier. The first failure throws `ExceptionInInitializerError`, and every later use of the class throws `NoClassDefFoundError`. Find the first error in the logs, not the repeated one.

Probe 3 (senior): You get `NoSuchMethodError` after upgrading one dependency. How do you diagnose and prevent it?

Expected reasoning: the compiled code called a method that the version loaded at run time does not have. Run `mvn dependency:tree` to find the competing versions, pin one in `dependencyManagement` or exclude the other, and add the Enforcer plugin's dependency convergence rule so it fails the build next time.

Tier 4 - Deep dive

Internals: loaders delegate to their parent first, so the first copy found along the delegation path wins. Servlet containers and plugin systems often change that order, which can load a different version than you expect. `ServiceLoader` finds providers through `META-INF/services` files, or provides in `module-info.java`. Building a

shaded jar can silently drop providers, because several jars contribute a file with the same path, so merge them, for example with Maven Shade's `ServicesResourceTransformer`.

Realistic failure: two JSON libraries each register a provider, and the shaded jar keeps only one service file. The application starts, but the missing provider never loads, and behavior differs from the unshaded build. It appears only in the packaged artifact.

Trade-off: pinning versions gives reproducibility and takes ongoing maintenance, and excluding a transitive dependency risks removing something a library needs. Shading avoids conflicts, at the cost of larger artifacts and merge details.

How to verify: print the actual source of a class with `SomeClass.class.getProtectionDomain().getCodeSource()` to see which jar was loaded. Run the Enforcer convergence rule in CI. Test the packaged artifact, not only the build directory.

Operations

Q291 How do you design alerts that wake people for the right reasons

Source basis: [Google SRE Workbook: Alerting on SLOs](#).

Tier 1 - Foundation

In plain language: an alert should tell a person that users are being hurt and that action is needed. Alerts that fire for harmless internal fluctuations teach people to ignore them, so the real ones get missed.

Key terms:

- **SLI (service level indicator):** a measurement of user experience, such as the fraction of successful requests.
- **SLO (service level objective):** the target for an SLI, such as 99.9% over 30 days.
- **Error budget:** the allowed failure, which is 100% minus the SLO.
- **Burn rate:** how fast the error budget is being consumed relative to plan.
- **Symptom / cause:** what users feel versus the internal reason.
- **Runbook:** written steps for responding to an alert.

Small example: an alert that fires whenever CPU exceeds 80% wakes someone every night and rarely indicates a user problem. An alert that fires when the success rate drops below the SLO at a damaging pace does.

Tier 2 - Core answer (30-second version)

Alert on symptoms users feel, such as errors and latency, not on every internal cause. Define an SLI and SLO and treat the remainder as an error budget, then page on burn rate, using a long and a short window together to cut noise and reset quickly. Every page should be actionable, link a runbook and have an owner. Keep cause-level metrics on dashboards for diagnosis, and remove alerts that fire without needing action.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why alert on symptoms rather than causes?

Expected reasoning: many causes never reach users, and some user-facing problems have causes you did not anticipate. A symptom alert catches both. Cause metrics help the responder after the page.

Probe 2 (applied): What does a burn rate of 14.4 mean for a 30-day SLO?

Expected reasoning: the error budget is being consumed 14.4 times faster than the rate that would use it all in exactly 30 days. Sustained for one hour, that consumes about 2% of the budget ($14.4 / 720$ hours), which justifies a page.

Probe 3 (senior): How do you alert on a low-traffic service?

Expected reasoning: with few requests, one failure can swing the ratio and cause noise. Use longer windows, synthetic probes that generate steady traffic, or alert on absolute error counts alongside ratios.

Tier 4 - Deep dive

Internals: for a 99.9% SLO the budget is 0.1%. A burn rate of 1 means errors at 0.1%, using the budget exactly over the window. The workbook's illustrative thresholds for a 30-day window are a burn rate of about 14.4 over one hour (about 2% of the budget) as a page, about 6 over six hours (about 5%) as a page, and about 1 over three days (about 10%) as a ticket. Each pairs a long window with a short one, commonly one-twelfth the length, so the alert stops soon after the problem does.

Realistic failure: a team pages on pod restarts and CPU. Most pages are self-healing, engineers start muting the channel, and a real regression in checkout success goes unnoticed for an hour because nobody trusts the pager.

Trade-off: fast detection means more false alarms, and conservative thresholds miss short incidents. Burn-rate alerts balance detection speed against noise but depend on a good SLI. A poor SLI, such as one that excludes failing paths, makes the alert useless.

How to verify: replay a past incident against the alert rules and check that it would have paged at the right time. Inject errors in staging and confirm the burn-rate alert fires and clears. Review each page in a regular session and fix or remove noisy ones.

Q292 How do you manage secrets for services on GKE

Tier 1 - Foundation

In plain language: secrets such as database passwords and API keys must reach the application without being stored in code, images or plain config files. A service should prove who it is to a secrets store, receive only what it needs, and have credentials that can be replaced without a rebuild.

Key terms:

- **Secret store:** a managed service such as Google Secret Manager.
- **Workload identity:** a platform identity for a pod, so it needs no stored key file.
- **Least privilege:** granting only the access that is needed.
- **Rotation:** replacing a credential with a new one on a schedule or after exposure.
- **Kubernetes Secret:** an object holding sensitive data, base64-encoded.

Small example: a service needs a database password. Instead of a key file in the image, its pod authenticates through workload identity and reads the password from the secret store.

Tier 2 - Core answer (30-second version)

Keep secrets out of Git, images and plain config. Store them in a managed secret store, give each workload its own identity with Workload Identity so it needs no long-lived key file, and grant only the secrets it needs. Kubernetes Secrets are base64, not encryption, so rely on access control and

encryption at rest. Plan rotation from the start with secret versions and clients that can pick up new values, and have a tested process for a leaked credential.

Tier 3 - Follow-ups

Probe 1 (warm-up): Is a Kubernetes Secret encrypted?

Expected reasoning: base64 is an encoding, not protection. Secrets are protected by who can read them (RBAC) and by encryption at rest on the cluster, so restrict access and confirm how your cluster stores them.

Probe 2 (applied): Why avoid service-account key files?

Expected reasoning: a key file is a long-lived credential that can be copied, committed or leaked, and must be rotated manually. Workload Identity exchanges the pod's identity for short-lived tokens, so there is nothing static to steal.

Probe 3 (senior): A credential leaks. What do you do?

Expected reasoning: revoke or disable it first, issue a new one and roll it out, then investigate where it was used with audit logs. Rotation that was never exercised before is a common reason this takes hours.

Tier 4 - Deep dive

Internals: Workload Identity Federation for GKE maps a Kubernetes service account to an IAM identity, and the workload obtains short-lived credentials through the platform. Secrets can be mounted as files with the Secret Manager CSI driver, or read through a client library or framework integration such as Spring Cloud GCP, whose current options you should check in its documentation. Secret versions allow staged rotation: create a new version, update consumers, then disable the old one. For databases, a smooth rotation often means two valid credentials for a window.

Realistic failure: a key file is committed to a repository and found by a scanner. Deleting the file does not help, because history keeps it, so the key must be revoked and replaced. If rotation has never been rehearsed, the service goes down when the credential is revoked.

Trade-off: environment variables are easy but can leak through crash dumps, debug output and child processes, while mounted files with tight permissions reduce that exposure. Spring Boot 3 masks values in the env and configprops actuator endpoints by default, but verify your configuration and keep those endpoints restricted.

How to verify: rotate a database password in a non-production environment and confirm the service recovers without manual steps. Try to read a secret from a workload that should not have access and confirm it is denied. Review audit logs to confirm access is recorded.